



— MODULE 03

JavaScript Foundations

| Make the Page Think



LEA LABS



The Browser Runs JavaScript

Browsers read HTML, apply CSS, and run JavaScript to make pages work.

HTML

Structure & Meaning

The semantic skeleton browsers and assistive tech read.

```
<button  
id="enroll">Join</button>
```

CSS

Presentation & Layout

Controls layout, typography, color, and responsive rules.

```
button { background: #f47945;  
color: white; }
```

JAVASCRIPT

Behaviour & Logic

Handles interactions, state, and server communication.

```
btn.addEventListener('click', ()  
=> enrollStudent());
```

Load Scripts Safely

JavaScript normally blocks HTML parsing. Use `defer` to download scripts early and run them after the page is built.

THE MODERN STANDARD

```
<head><script src="app.js" defer>  
</script></head>
```

Putting a script in the `<head>` with `defer` downloads early but runs after the DOM is ready, avoiding "element not found" errors.

WITHOUT DEFER (BLOCKING)



The user sees a blank screen while JS loads.

WITH DEFER (NON-BLOCKING)



The user sees the page immediately.

Values and Variables

Variables are named containers for data. Modern JavaScript uses two keywords to declare them, communicating your intent to other developers.

const

THE DEFAULT CHOICE

Creates a constant reference to a value. The variable cannot be reassigned later. Using `const` makes your code predictable and easier to read.

```
const programmeName = "Software  
Engineering";  
const durationWeeks = 16;  
  
// This will throw an error:  
durationWeeks = 20; TypeError
```

let

FOR CHANGING VALUES

Creates a variable that can be reassigned. Use `let` only when you know the value will change, such as in loops or when tracking user input.

```
let enrolledStudents = 0;  
  
// Later, when someone joins:  
enrolledStudents = 1;  
enrolledStudents = 2; // Valid
```

Strings, Numbers, Booleans

Every piece of data in your interface maps to a specific JavaScript type. Strings hold text, numbers hold math-ready values, and booleans hold true/false states.

STRING (TEXT)

```
const courseName =  
"Software  
Engineering";
```

NUMBER (MATH)

```
const priceKES =  
45000;
```

BOOLEAN (LOGIC)

```
const isEnrolled = true;
```

ENROLLED`isEnrolled === true`

Software Engineering

KES 45,000`typeof Number`

Conditions Choose Behaviour

Programs must make decisions based on data. We use `if` and `else` statements to execute different code depending on whether a condition is true or false.

```
const isEnrolled = true;
const seatsLeft = 0;

let statusMessage;

if (isEnrolled) {
  statusMessage = "You are enrolled.";
} else if (seatsLeft > 0) {
  statusMessage = "Seats available!";
} else {
  statusMessage = "Course full.";
}
```

CONDITION 1 (TRUE)

You are enrolled.

CONDITION 2 (SKIPPED)

Seats available!

FALLBACK (SKIPPED)

Course full.

Functions Package Actions

Functions group statements to perform a task. Write logic once and reuse it to keep code predictable.

1. DEFINE THE ACTION

You declare a function to teach the browser a new command. It waits until you call it.

2. PASS INFORMATION IN

Functions accept **parameters**—placeholders for the data you provide later.

3. RETURN THE RESULT

A function should compute a value and **return** it, rather than modifying the page. This makes it reusable.

FORMATTING A COURSE PRICE

// Definition

```
function formatPrice(amount){ const
formatted=amount.toLocaleString(); return
`KES ${formatted}` }
```

// Execution

```
const sePrice=formatPrice(45000),
aiPrice=formatPrice(35000)
console.log(sePrice, aiPrice) // "KES
45,000" "KES 35,000"
```

Arrays Hold Sequences

When you have multiple related items—like a list of programmes—you store them in an array. Arrays keep items in a specific, numbered order.

ORDERED LISTS

```
const programmes = [ 'Software  
Engineering', 'Applied AI', 'Basic  
Computer' ];  
console.log(programmes[1]);  
programmes.push('Data Science');
```

0	"Software Engineering"
1	"Applied AI"
2	"Basic Computer"
3	"Data Science" (pushed)

Objects Describe One Thing

While arrays hold lists of similar items, objects group related properties about a single entity. They use key-value pairs to structure complex data clearly.

DEFINING AND ACCESSING

```
const learner = {  
  name: "Alice",  
  programme: "Software Engineering",  
  modulesCompleted: 3,  
  isActive: true  
};
```

```
// Read data using Dot Notation  
console.log(learner.name);  
// "Alice"
```

```
// Update existing properties  
learner.modulesCompleted = 4;
```

LEARNER

OBJECT

name	"Alice"
programme	"Software Engineering"
modulesCompleted	3
isActive	true

Loops Repeat Deliberately

Instead of writing the same code multiple times, loops allow you to execute a block of code for every item in a collection. The `for . . . of` loop is perfect for arrays.

PROCESSING AN ARRAY

```
const programmes = [  
  'Software Engineering',  
  'Applied AI',  
  'Basic Computer'  
];  
  
// For each item in the array...  
for (const prog of programmes) {  
  // ...run this code block  
  console.log(`Render: ${prog}`);  
}
```

Loop 1	prog = 'Software Engineering'
--------	-------------------------------

Loop 2	prog = 'Applied AI'
--------	---------------------

Loop 3	prog = 'Basic Computer'
--------	-------------------------

Errors Become Useful Feedback

Code will fail—networks drop, data is malformed, or inputs are invalid. Instead of letting the application crash, we anticipate failure and handle it gracefully.

TRY { ... }

Wrap risky operations here. If everything works, the code executes normally. If an error occurs, execution immediately jumps to the catch block.

CATCH (ERROR) { ... }

The safety net. This block only runs if the try block fails. It receives an error object, allowing you to log the issue and show a friendly message to the user.

SAFE DATA PARSING

```
const badData = "This is not JSON";
try {
  const parsed = JSON.parse(badData);
  console.log("Success!", parsed);
} catch (error) {
  console.error("Parse failed:",
    error.message);
  showErrorBanner("Unable to load data.");
}
```

Build a JS-Powered Summary

Apply your foundational JavaScript knowledge to process an array of course objects, calculate totals, and generate a dynamic summary report.

DELIVERABLES

- **Data** Array of ≥ 3 course objects (title, price, **Structure:** isEnrolled).
- **Iteration:** Use `for...of` to count enrolled courses.
- **Logic:** Function returns total price of enrolled courses.
- **Error Handling:** Wrap calculations in `try/catch`.

SUCCESS CRITERIA

- Console logs correct total and enrolled count.
- No runtime or syntax errors occur.
- Use `const` and `let` appropriately.

EXTENSION CHALLENGE

Add a helper to format the total as currency (e.g., "KES 45,000").